



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Technical presentation

CSA1709- Artificial Intelligence

Presented By
X.Joshua-(192312188)

Guided By
Dr. P Raja ram



SIMATS ENGINEERING

CO5-ASSESSMENT TOOL3 - Reverse Engineering



COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

Register Number	X. Joshua
Name	192312188

COURSE OUTCOME COVERED

CO5: Design intelligent agents and real-time AI-based systems (chatbots, expert systems, emotion detection, edge AI, autonomous drones, and related applications) by selecting appropriate agent architectures, knowledge representation, and reasoning mechanisms to solve real-world problems. (BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Q1. The following is a conversation between a user and an AI-based flight-booking chatbot. Study the transcript given below and reverse-engineer the chatbot's design by: (a) writing its PEAS description, (b) classifying the type of agent it represents with justification, (c) identifying the likely component (NLU, Dialogue Manager, Knowledge Base, or NLG) responsible for each chatbot response, and (d) suggesting one enhancement using a learning-agent architecture.

Speaker	Message
User	I want to book a flight to Chennai next Friday.
Chatbot	Sure! What time would you like to depart, and do you have a preferred airline?
User	Morning flight, no preference.
Chatbot	I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first?

Q2. The block diagram below shows the processing pipeline of a Real-Time Emotion Detection System. Reverse-engineer the system by: (a) explaining the function performed at each stage of the pipeline, (b) identifying the likely AI/ML technique used at each stage, (c) writing the PEAS description of the system, and (d) classifying the agent type (e.g., simple reflex, model-based) with justification.

Real-Time Emotion Detection System — processing pipeline



Q3. An Expert System Shell for medical diagnosis contains the rule base, input facts, and reasoning trace given below. Study it and reverse-engineer the shell by: (a) identifying the type of chaining (forward/backward) used to



Overall system verdict: Fake (3 of 5 frames flagged — majority vote)

Code of Conduct:

I X. Joshua (192312188) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

X. Joshua

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Identification of Agent Type & PEAS (correctly inferred from the given artifact)	Correctly identifies the agent type and gives a complete, accurate PEAS description, fully consistent with the given artifact.	Identifies the agent type and PEAS description correctly with only minor omissions.	Partially correct identification of agent type/PEAS; some elements are missing or inconsistent with the artifact.	Agent type/PEAS identification is largely incorrect or not attempted.	10
Identification of Architecture & Components (mapping artifact evidence to system components)	Accurately identifies all relevant architectural components and correctly maps each to specific evidence in the artifact.	Identifies most components correctly with reasonable mapping to the artifact; minor gaps present.	Identifies some components; mapping to the artifact is weak or partially incorrect.	Little or no correct identification of architecture/components.	10
Identification of Underlying Technique/Algorithm/Reasoning Mechanism	Correctly reverse-engineers the technique, algorithm, or reasoning mechanism (e.g., chaining type, ML technique) with clear supporting evidence.	Correctly identifies the technique/mechanism with adequate, mostly accurate justification.	Identification is only partially correct or lacks sufficient supporting evidence.	Technique/mechanism is incorrectly identified or not addressed.	10
Quality of Justification & Use of Evidence (linking conclusions back to the given artifact)	Every conclusion is clearly justified with specific, accurate references to the given artifact.	Most conclusions are justified with reasonable references to the artifact.	Justification is vague or only loosely connected to the artifact.	Conclusions are stated with little or no justification or evidence.	10
Clarity & Completeness of Presentation (addresses all sub-parts of the question in an	All sub-parts are addressed completely, in a clear, well-organised, and logically	Most sub-parts are addressed clearly with minor gaps in organisation	Some sub-parts are missing or the presentation is unclear/poorly organised.	Response is incomplete, disorganised, or largely missing.	10



Introduction



- Reverse engineering is the process of studying an existing system to understand its architecture, components, processing methods, and output.
- In an AI-based system, reverse engineering helps identify how input data is converted into meaningful predictions.
- The analysis focuses on the input, preprocessing, feature extraction, AI model, decision-making, and final output.
- The selected example is an AI-based Deepfake and Face Authenticity Verification System.
- The system analyzes facial information from images or videos and determines whether the content is genuine or manipulated.



Problem Statement

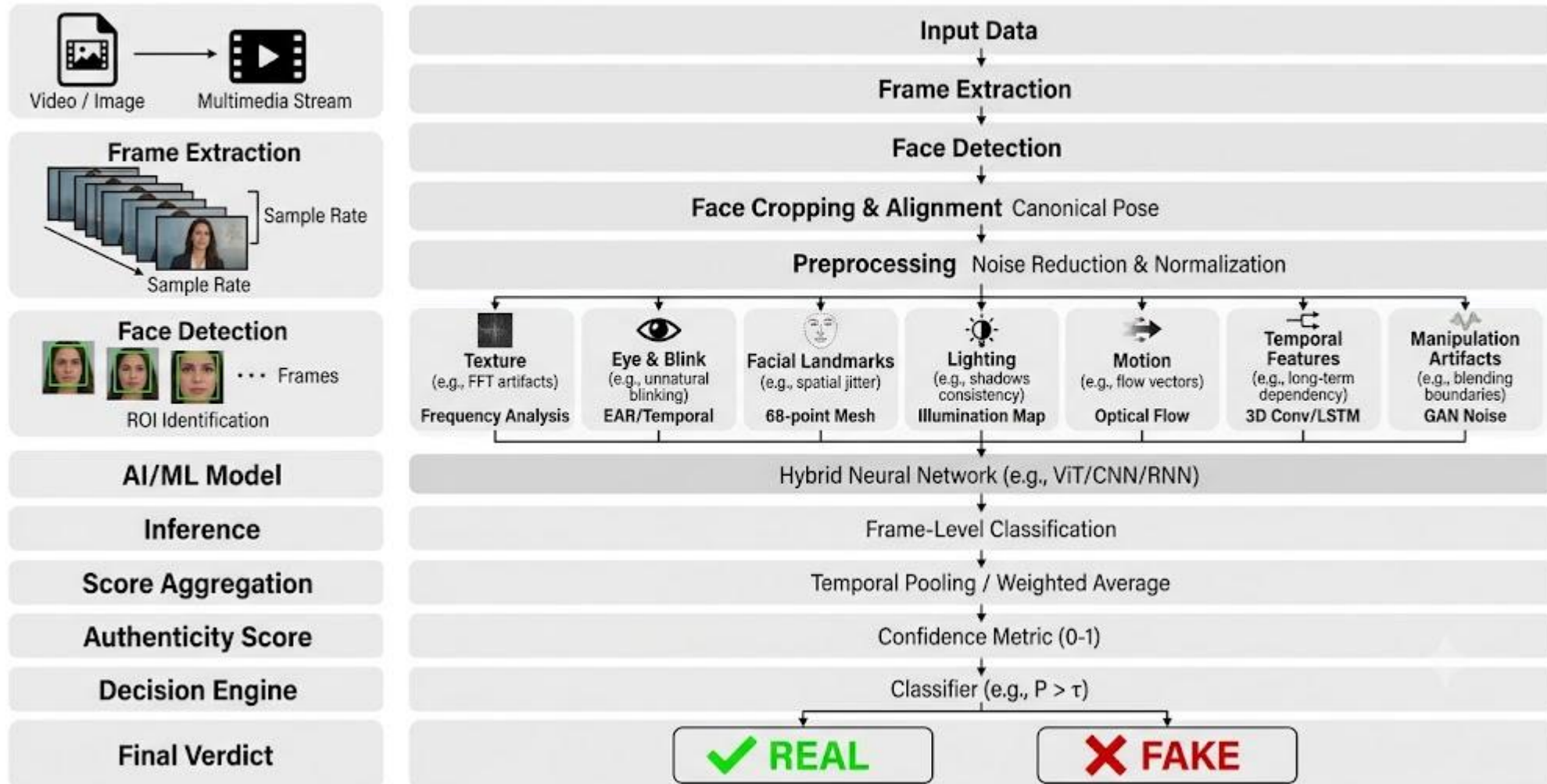
- The main objective is to automatically determine whether a given facial image or video is REAL or FAKE.
- Detect artificial modifications introduced by deepfake or face-manipulation techniques.
- Analyze facial appearance, movement, texture, and temporal consistency.
- Generate a probability or confidence score for the prediction.
- Provide a final authenticity decision that can assist users or security systems.
- Reduce the need for completely manual inspection of suspicious videos.



architecture



DEEPPFAKE DETECTION PIPELINE ARCHITECTURE





System Output



1. The final output contains the authenticity classification.
2. The system may display **REAL** or **FAKE**.
3. It can also provide a confidence or probability score.
4. Suspicious frames can be identified for further investigation.
5. Detected artifacts or features may be included in the verification report.
6. The output can be displayed through a web application, mobile application, or security dashboard.
7. The result can help a human operator make the final verification decision.



Advantages



1. Provides automated deepfake detection.
2. Reduces manual video inspection.
3. Can process multiple frames efficiently.
4. Can combine different facial features.
5. Can identify subtle visual inconsistencies.
6. Provides measurable confidence scores.
7. Can be integrated into online identity verification systems.
8. Can support cybersecurity and digital forensics.
9. Can potentially operate in real-time with suitable hardware and optimized models.



Conclusion



1. Reverse engineering allows us to understand an AI-based system by analyzing its observable behavior.
2. The deepfake verification system can be divided into input, preprocessing, feature extraction, AI classification, score aggregation, and decision-making stages.
3. The system likely analyzes both spatial features and temporal features to identify manipulated facial content.
4. Multiple frame predictions can be combined to produce a more reliable video-level result.
5. The final system provides a REAL/FAKE classification along with a confidence or authenticity score.
6. Such systems are increasingly useful for digital identity verification, cybersecurity, digital forensics, and online authentication.



Introduction

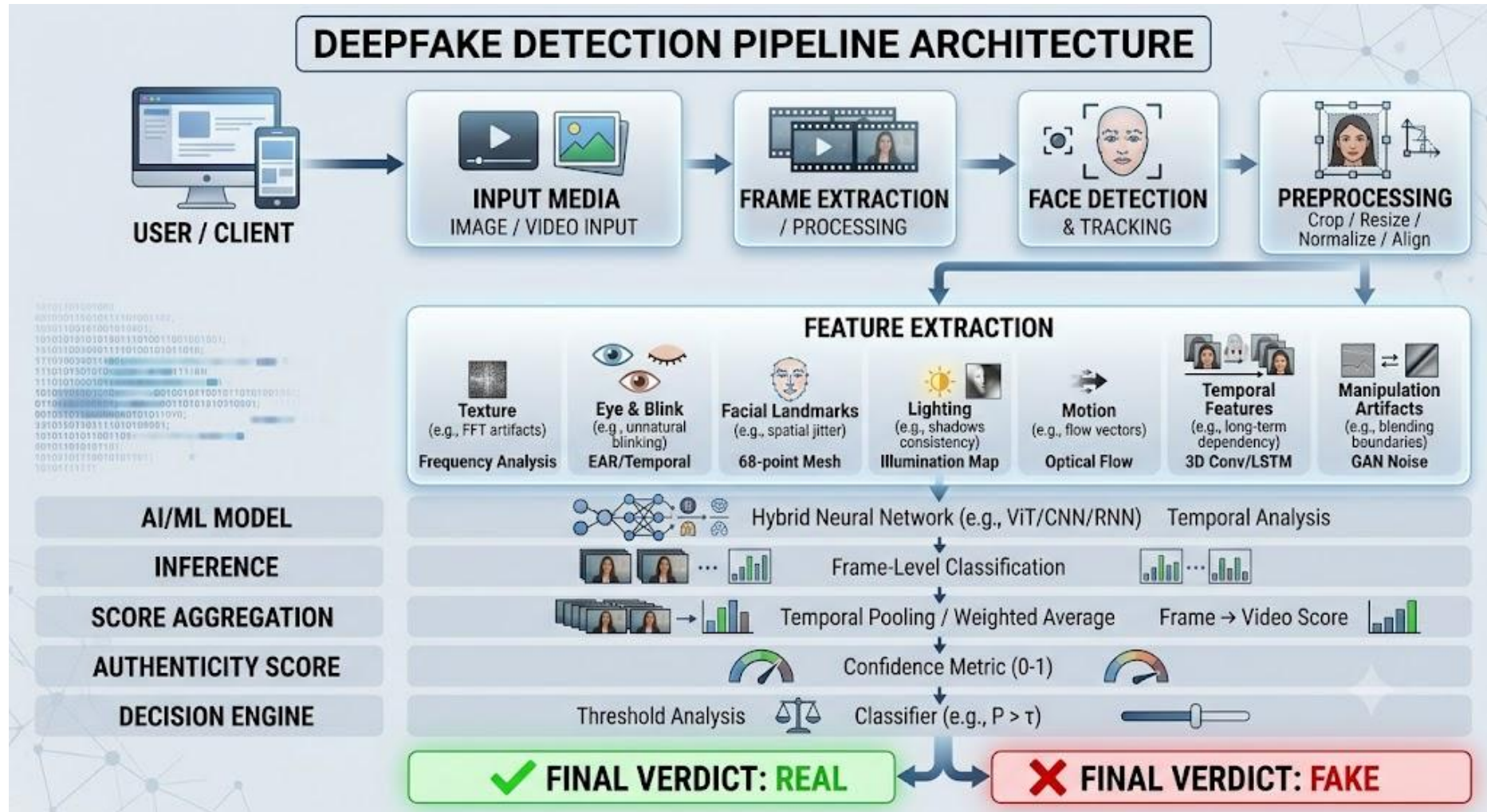
1. Architecture reconstruction is the process of analyzing an existing software or AI system to understand its internal structure.
2. It identifies the major modules, their responsibilities, interactions, and data flow.
3. The process helps understand how input data is transformed into the final output.
4. In this system, the architecture of an **AI-based Deepfake and Face Authenticity Verification System** is reconstructed.
5. The system uses image/video processing and AI techniques to identify whether facial content is genuine or manipulated.
6. The reconstructed architecture provides a clear view of how different modules work together.



Problem Statement

1. The rapid growth of AI-generated and manipulated images/videos has made it difficult to distinguish genuine facial content from deepfake content.
2. Deepfake videos can realistically modify a person's face, expressions, eye movements, skin texture, and facial features.
3. Manual verification of videos is time-consuming and may not reliably identify sophisticated manipulations.
4. Therefore, an AI-based Face Authenticity Verification System is required to automatically analyze images or videos.
5. The system should extract relevant facial features, detect manipulation artifacts, analyze frame-to-frame consistency, and classify the input as REAL or FAKE.
6. The system should also provide an authenticity/confidence score to support the final verification decision.
7. The reconstructed architecture should clearly represent the modules, interactions, and data flow involved in the verification process.

architecture





System Output



- The system produces a final REAL / FAKE classification for the input image or video.
- It generates an authenticity or confidence score indicating how strongly the system supports the prediction.
- Provides frame-level prediction scores for individual video frames.
- Displays detected facial artifacts and inconsistencies that may indicate manipulation.
- Reports abnormalities in:
 - Facial texture
 - Eye and blink movement
 - Facial landmarks
 - Lighting consistency
 - Face boundaries
 - Temporal/frame consistency
- Combines individual frame results to generate an overall video authenticity score.
- The final result can be displayed through a web interface, dashboard, or verification report.



Advantages



1. Provides a clear understanding of the complete AI system.
2. Divides the system into independent and manageable modules.
3. Makes data flow between components easy to understand.
4. Supports automated deepfake detection.
5. Can analyze multiple frames instead of relying on a single image.
6. Combines different facial features for better analysis.
7. AI models can be improved or replaced independently.
8. Provides authenticity scores along with the final decision.



Conclusion



- Architecture reconstruction helps identify how an existing AI system operates internally.
- The reconstructed system consists of **input processing, frame extraction, face detection, preprocessing, feature extraction, AI analysis, score aggregation, and decision-making.**
- Each module performs a specific function and passes processed information to the next module.
- The AI model analyzes facial and temporal features to identify possible manipulation.
- The final system produces a **REAL or FAKE** result with an authenticity/confidence score.
- Thus, architecture reconstruction provides a clear technical understanding of the system's **components, interactions, and data flow.**



Introduction



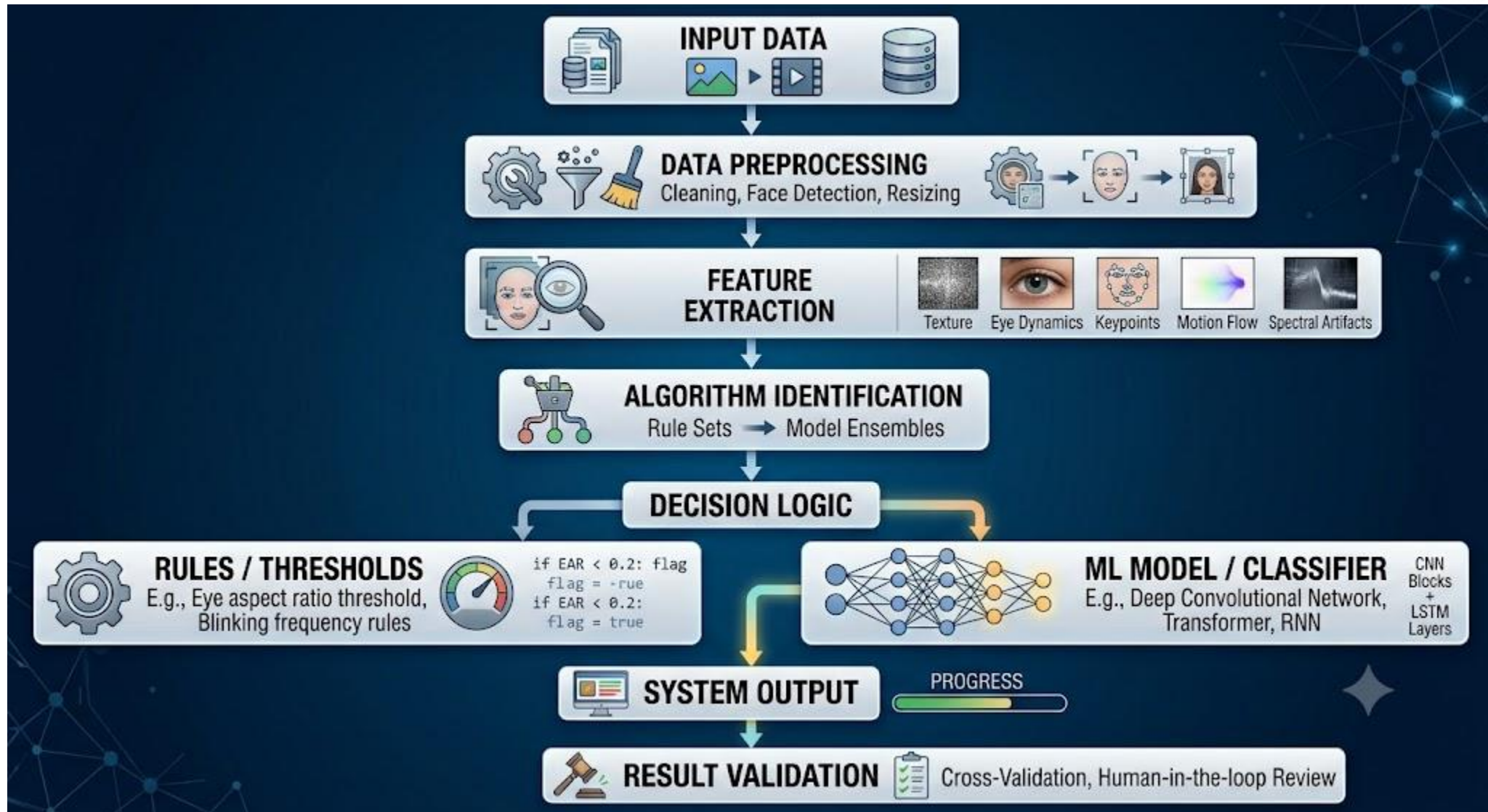
- Algorithm and Logic Identification is the process of analyzing a software or AI system to determine the algorithms, rules, conditions, and decision-making methods used internally.
- The system is treated as a black box, where the internal source code may not be available, so its behavior is studied using inputs, outputs, logs, and repeated testing.
- The main objective is to understand how the system transforms input data into a final result.
- Different inputs are provided to the system and the corresponding outputs are carefully observed to identify common patterns and processing logic.
- This process helps determine whether the system is using rule-based logic, machine learning, classification, regression, pattern matching, or optimization techniques.



Problem Statement

- The major problem is to identify the hidden algorithm and decision-making logic of an existing software or AI system without depending completely on its original source code.
- The system may contain multiple processing stages, making it difficult to determine which algorithm is responsible for a particular output.
- Different input conditions must be tested to understand how the system reacts to normal, abnormal, and boundary cases.
- Important features used by the system must be identified by observing which input values have a strong effect on the final output.
- Decision rules, thresholds, conditions, confidence scores, and classification boundaries must be analyzed to reconstruct the decision-making process.
- The identified algorithm must be tested against additional inputs to verify whether the reconstructed logic produces results similar to the original system.

architecture





System Output



1. The system produces a final output after processing the input through preprocessing, feature extraction, algorithm execution, and decision-making stages.
2. The output can contain a prediction, classification, probability, confidence score, numerical value, recommendation, or final decision.
3. The output is analyzed together with the original input to determine which features and rules influenced the result.
4. Multiple outputs are compared to identify whether the system behaves consistently under different conditions.
5. Example:
 1. Input: Image or data sample
 2. Extracted Feature Score: 0.82
 3. Model Prediction: Positive
 4. Confidence Score: 94%
 5. Decision: Accepted



Advantages



1. Helps understand the internal behavior of systems whose source code is unavailable or difficult to analyze.
2. Helps identify the **most likely algorithm, model, rules, and decision-making techniques** used by the system.
3. Makes it possible to reproduce the behavior of an existing system using a newly implemented algorithm.
4. Helps developers understand how different modules interact during data processing.
5. Supports debugging by identifying the stage where an incorrect decision or output is generated.
6. Helps identify hidden thresholds and decision boundaries that affect system results.
7. Supports testing and validation of AI and machine-learning systems.
8. Helps organizations document old or poorly documented software systems.



Conclusion



1. Algorithm and Logic Identification provides a systematic approach for reverse-engineering the decision-making process of a software or AI system.
2. By studying inputs, outputs, features, logs, patterns, thresholds, and decision boundaries, the hidden processing logic can be reconstructed.
3. Black-box testing and input-output analysis are especially useful when the original source code is unavailable.
4. Tools such as Python, Jupyter Notebook, debugging tools, and log analysis techniques help verify the identified algorithms.
5. The reconstructed architecture and logic can be used for system validation, debugging, documentation, maintenance, modernization, and performance improvement.



Introduction



1. Data Flow and Processing Pipeline explains how data moves from the **source system to the final output** in a reverse-engineered distributed application.
2. The system is analyzed by identifying how data is **collected, transmitted, processed, stored, and returned**.
3. In a distributed system, different services or modules may process different parts of the data.
4. Communication between these modules can occur through **APIs, message queues, WebSockets, databases, or network protocols**.
5. Reverse engineering helps reconstruct the complete data journey even when the original architecture or source code is unavailable.



Problem Statement

- Identify where the data originates and determine the different sources providing information to the system.
- Determine how data travels between different modules, servers, services, and processing components.
- Identify the preprocessing operations performed before the main computation.
- Determine how data is distributed among multiple processing nodes.
- Identify the algorithms or services responsible for processing the received data.
- Analyze how intermediate results are transferred between different components.
- Identify the communication mechanisms used between distributed modules.



architecture





System Output



1. The system receives data from the user or external data sources and processes it through multiple distributed modules.
2. After preprocessing and validation, the data is distributed to different processing nodes for parallel execution.
3. Each processing node generates an intermediate result based on its assigned task.
4. The intermediate results are transferred through APIs, message queues, WebSockets, or other communication mechanisms.
5. A result aggregation module combines the outputs received from different processing nodes.
6. The final processed result is stored in the database or distributed storage system.
7. The completed result is then returned to the user through the application interface or API.



Advantages



1. Provides a clear understanding of how data travels through the distributed system.
2. Helps identify the responsibilities of individual processing modules.
3. Makes it easier to locate communication bottlenecks and processing delays.
4. Supports identification of APIs, queues, databases, and network communication methods.
5. Helps understand how large workloads are divided among multiple processing nodes.
6. Improves understanding of parallel and distributed processing.
7. Helps identify failures in data transmission and processing.
8. Supports system debugging and performance optimization.
9. Makes legacy distributed systems easier to document and maintain.



Conclusion



1. Data Flow and Processing Pipeline analysis provides a complete view of **how information enters, moves through, and exits a distributed system.**
2. Reverse engineering identifies the major processing stages, communication mechanisms, distributed nodes, storage systems, and final output generation.
3. By reconstructing the pipeline, the interaction between different modules can be clearly understood.
4. This analysis is useful for **debugging, documentation, performance optimization, scalability, system modernization, and architecture reconstruction**



Introduction



1. Enhancement and Redesign Proposal focuses on improving an existing software or AI system after analyzing its current architecture, algorithms, data flow, and limitations.
2. Reverse engineering helps identify **performance issues, outdated components, security weaknesses, scalability problems, and inefficient processing.**
3. The proposed redesign aims to make the system **faster, more secure, scalable, reliable, maintainable, and easier to upgrade.**
4. Existing functionality is preserved while inefficient modules are replaced or improved.
5. The redesigned architecture is based on the actual problems identified during reverse engineering

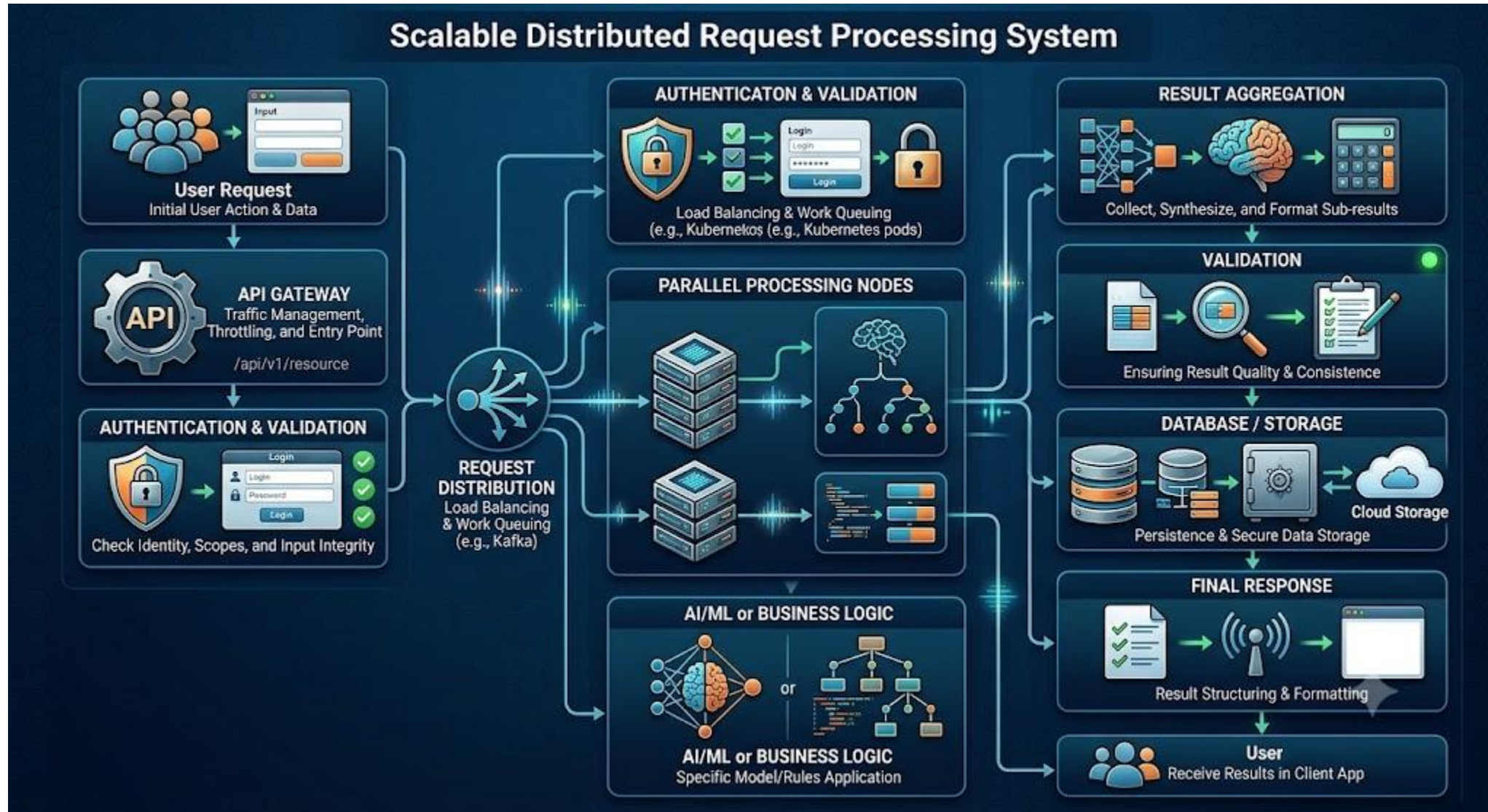


Problem Statement

1. The existing system may contain tightly coupled modules that make maintenance difficult.
2. Data processing may be slow because tasks are performed sequentially instead of in parallel.
3. Communication between modules may create unnecessary delays.
4. The system may have limited scalability when the number of users or requests increases.
5. Old algorithms may require excessive processing time or computational resources.
6. Database queries and storage operations may become bottlenecks.
7. Security mechanisms may need improvement to protect sensitive data.
8. Error handling and monitoring may be insufficient.
9. Lack of modular architecture can make future upgrades difficult.



architecture





System Output



1. The redesigned system produces **faster, more reliable, secure, and scalable outputs** compared with the existing system.
2. User requests are processed through the **API Gateway**, validated, authenticated, and forwarded to the appropriate services.
3. Large tasks are divided into smaller processing jobs and distributed across multiple worker nodes.
4. Independent processing tasks are executed **in parallel**, reducing overall processing time.
5. Results from different services and worker nodes are collected and combined by the result-processing module.
6. The final result is validated before being stored in the database or distributed storage.
7. The processed result is then returned to the user through the web application, mobile application, or API.



Advantages



1. Higher Performance: Optimized algorithms and parallel processing reduce processing time.
2. Better Scalability: Additional servers or workers can be added when workload increases.
3. Improved Security: Strong authentication, encryption, and access control protect system resources.
4. Higher Reliability: Fault-tolerant components prevent individual failures from affecting the complete system.
5. Easy Maintenance: Modular components can be modified independently.
6. Better Monitoring: Centralized logs and performance metrics make problems easier to identify.
7. Future Flexibility: New algorithms, services, and AI models can be integrated without rebuilding the complete system.



Conclusion



1. Reverse engineering provides detailed knowledge about the existing system's architecture, algorithms, data flow, communication mechanisms, and limitations.
2. Based on these findings, the proposed redesign introduces modular architecture, distributed processing, optimized databases, secure communication, fault tolerance, monitoring, and scalability.
3. The redesigned system can provide better performance, reliability, security, maintainability, and flexibility.
4. The enhancement strategy also prepares the system for future technologies, increased workloads, and new AI/ML capabilities.
5. Therefore, the proposed redesign transforms the existing system into a more efficient, scalable, secure, and future-ready architecture.